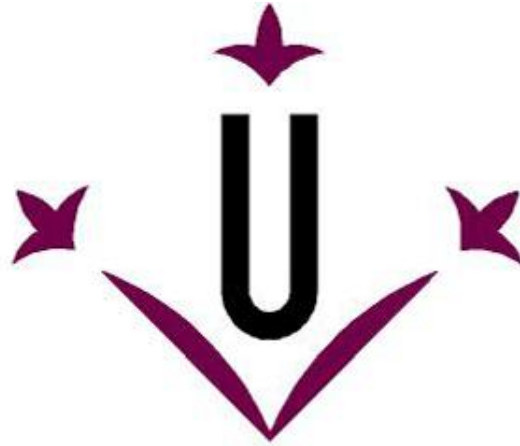




Universitat de Lleida

High Performance Computing
Convolution Operation CUDA Practice

Students:

Joel Castillo Estallo
David Carreras Castañ

Date: 06 June 2026

Main decisions taken to parallelize the problem

The CUDA implementation was developed starting from the original serial version of the convolution program, keeping its input/output structure and replacing only the most computationally expensive part of the program with a GPU implementation.

As in the previous deliveries, the dominant phase of the application is the convolution itself. The serial program reads the image, reads the convolution kernel, applies the convolution independently to the three colour channels, and stores the resulting image. Since the convolution clearly represents the main computational hotspot, the CUDA version was designed by keeping the file reading and writing stages on the CPU and moving only the convolution stage to the GPU. This decision simplifies the implementation and makes the CUDA design easier to validate, while still accelerating the most expensive phase of the program.

At the GPU level, the parallelization strategy is based on assigning **one CUDA thread to one output pixel**. The convolution kernel is launched over a two-dimensional grid of thread blocks, so that each thread computes the value of one pixel in the output image. Since the image is stored as three independent channels (R, G, and B), the same CUDA kernel is launched three times: once for each channel. This keeps the implementation simple and close to the structure of the original serial code, while still exploiting the large amount of data parallelism available in the convolution operation.

Another important design decision was to make the CUDA block dimensions configurable through command-line arguments. Instead of fixing a single block size in the source code, the program accepts blockX and blockY as execution parameters.

Finally, the timing methodology was adapted to the CUDA execution model. Instead of measuring only the total execution time, the CUDA version separates the main phases of the execution: image reading, kernel reading, host-to-device memory transfers, kernel execution on the GPU, device-to-host memory transfers, result writing, and total time. This phase-based timing is important for CUDA because GPU performance depends not only on the arithmetic computation itself, but also on the cost of moving data between CPU and GPU memory. This makes the CUDA report directly comparable with the previous deliveries, while also reflecting the specific characteristics of GPU execution.

The source code can be found in the GitHub repository:

<https://github.com/JoelCastEst/HPC-OMP-Project>

The file executions.txt contains all the commands used to run the CUDA experiments. Each command specifies the input image, the convolution kernel, the output image file, the partitions argument kept for compatibility, and the CUDA block dimensions (blockX, blockY) used in each test.

Alternatives considered

Several alternatives were considered before fixing the final CUDA design. First, a multi-GPU implementation was considered, but it was discarded because the assignment does not require it and it would have introduced additional complexity related to image partitioning between devices, memory management on multiple GPUs, and coordination of results. For this reason, a single-GPU solution was selected. In addition, we considered fixing the CUDA block dimensions directly in the source code, but this was rejected because the assignment explicitly asks to test different Grid and Thread size configurations. Therefore, blockX and blockY were provided as command-line arguments, which made the evaluation of different CUDA configurations much more flexible.

Scalability of the Program

The scalability of the CUDA implementation was evaluated by measuring the execution time of the selected benchmark cases and comparing the obtained results with those from the previous versions of the program. Unlike the previous deliveries, in the CUDA version we also analysed the effect of different GPU execution configurations, since the assignment requires testing different Grid and Thread size configurations. For this reason, the scalability study was carried out not only for different image and kernel sizes, but also for several CUDA block configurations chosen during the experimental phase, such as 8x8, 16x16, and 32x8.

The standard formulas used were:

$$Speedup = \frac{T_{ref}}{T_{CUDA}} \qquad Efficiency = \frac{Speedup}{p}$$

Where T_{ref} is the execution time of the previous Serial, OMP and Hybrid versions, T_{CUDA} is the execution time of the CUDA version, and p represents the reference CUDA configuration used in the experiments (64, 256 and 256). In our case, p is interpreted from the selected thread-block configuration (8x8, 16x16 and 32x8), which allows the efficiency values to be used mainly as a normalized comparison metric between CUDA executions and previous CPU-based implementations.

Note that therefore, we will obtain 3 speedup values for each execution, relating CUDA execution with the 3 previous versions.

It is important to note that these CUDA efficiencies should not be interpreted in the same way as the classical efficiencies used in serial/OpenMP/MPI studies, where the number of CPU cores is clearly defined. In GPU computing, execution depends on many hardware and runtime factors, such as thread-block layout, warps, scheduling, memory hierarchy and device

occupancy. For this reason, the efficiency values reported here are used mainly to preserve consistency with the previous deliveries and to provide an additional relative indicator of how favorable each CUDA configuration is with respect to the serial, OpenMP and hybrid baselines, rather than as an absolute measure of GPU hardware utilization.

Tables 1 to 5 show the measured execution times for each image and each kernel. Note that all the concrete execution times can be found in the Outputs_CUDA folder located in the GitHub.

Image	Kernel			
Im01	kernel5x5_S harpen.txt	Kernel25x25_R andom2.txt	Kernel49x49_R andom2txt	Kernel99x99- Random2.txt
Serial	0.112099	0.641137	2.197561	8.308373
OMP 4 Threads	0.077999	0.352565	1.145758	4.280987
MPI 4 x OMP 4	0.823422	0.915604	1.319237	2.591515
CUDA 8x8	0.141184	0.143991	0.141316	0.151572
CUDA 16x16	0.144190	0.139648	0.141180	0.149004
CUDA 32x8	0.145025	0.143421	0.145649	0.151454

Table 1. Table with arguments to calculate the elapsed times of the Im01.

Image	Kernel			
Im02	kernel5x5_S harpen.txt	Kernel25x25_R andom2.txt	Kernel49x49_R andom2txt	Kernel99x99- Random2.txt
Serial	0.192128	1.621497	5.636850	22.308050
OMP 4 Threads	0.161151	0.871742	2.917048	11.133542
MPI 4 x OMP 4	1.186121	2.023907	3.118929	5.961890
CUDA 8x8	0.194375	0.189079	0.191520	0.199617
CUDA 16x16	0.187164	0.182986	0.193705	0.196339
CUDA 32x8	0.185089	0.188740	0.190952	0.199423

Table 2. Table with arguments to calculate the elapsed times of the Im02.

Image	Kernel			
Im03	kernel5x5_S harpen.txt	Kernel25x25_R andom2.txt	Kernel49x49_R andom2txt	Kernel99x99- Random2.txt
Serial	0.436407	3.317847	11.481928	46.686213
OMP 4 Threads	0.323869	1.729308	5.867542	22.389592
MPI 4 x OMP 4	1.974338	3.342716	5.222613	12.933911
CUDA 8x8	0.279828	0.276361	0.281368	0.299357
CUDA 16x16	0.278486	0.271050	0.279432	0.297565
CUDA 32x8	0.283171	0.277807	0.286177	0.300784

Table 3. Table with arguments to calculate the elapsed times of the Im03.

Image	Kernel			
Im04	kernel5x5_S harpen.txt	Kernel25x25_R andom2.txt	Kernel49x49_R andom2txt	Kernel99x99- Random2.txt
Serial	17.955136	162.369510	581.173065	2529.892722
OMP 4 Threads	14.464880	84.165687	289.594108	1139.728952
MPI 4 x OMP 4	84.661016	115.333869	214.577775	577.955941
CUDA 8x8	7.886144	7.813612	8.221394	8.883654
CUDA 16x16	7.882198	7.805843	8.215608	8.866340
CUDA 32x8	7.892132	7.808787	8.223509	8.855956

Table 4. Tble with arguments to calculate the elapsed times of the Im04.

Image	Kernel			
Im05	kernel5x5_S harpen.txt	Kernel25x25_R andom2.txt	Kernel49x49_R andom2txt	Kernel99x99- Random2.txt
Serial	65.472774	643.877559	2340.317637	10373.402927
OMP 4 Threads	50.160328	341.993654	1145.793739	4499.576473
MPI 4 X OMP 4	326.198191	444.432260	810.927725	2311.810490
CUDA 8x8	31.220197	31.005498	32.793468	36.156069
CUDA 16x16	31.364170	30.911927	32.576000	35.117181
CUDA 32x8	31.220719	30.896541	32.558962	35.144721

Table 5. Table with arguments to calculate the elapsed times of the Im05.

We decided to represent the time values in a table, as we consider, tables capture the exact information that plots sometimes can't. Even so, we decided to plot two overall graphics, for Speedup [Image 1] and for Efficiency [Image 2]

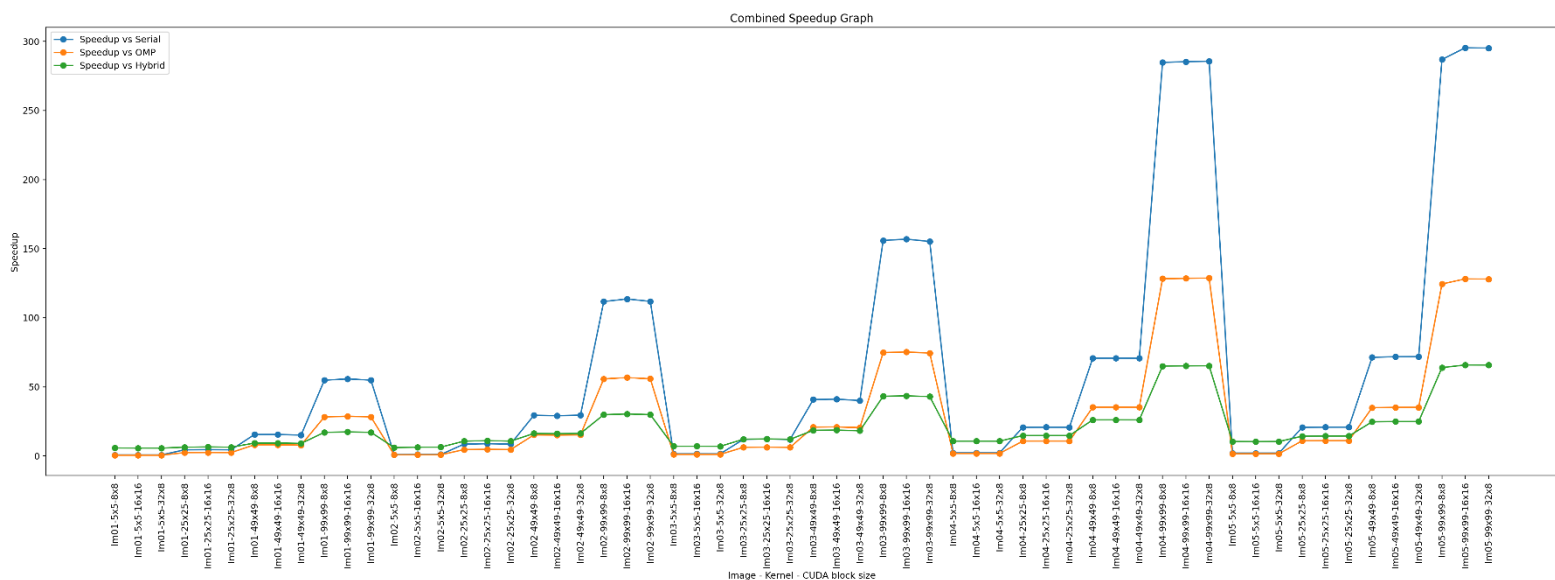


Image 1. Graph for overall Speedup

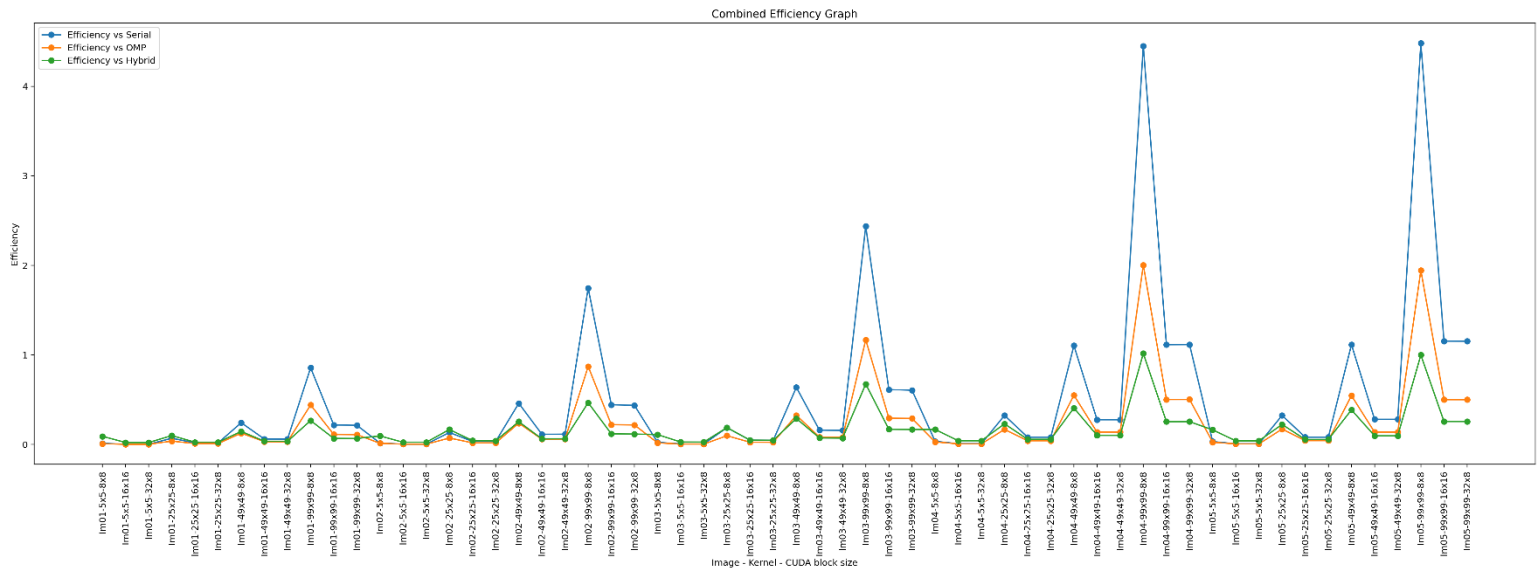


Image 2. Graph for overall Efficiency by Image, Kernel and Thread Count

The obtained results show that the CUDA version becomes increasingly advantageous as the workload grows. For the smallest cases, especially with the 5x5 kernel, the improvement over the CPU-based versions is limited, which suggests that GPU transfer and launch overheads still have a noticeable impact. However, as image size and kernel size increase, the CUDA implementation clearly outperforms the serial, OpenMP and hybrid versions, reaching its best behaviour in the most demanding cases, particularly with the 49x49 and 99x99 kernels. The three tested CUDA configurations (8x8, 16x16, and 32x8) show similar overall behaviour, although 16x16 and 32x8 tend to provide slightly better results in the largest workloads. Overall, the figures and tables indicate that the GPU-based solution is especially effective when the computational intensity is high enough to compensate for memory-transfer overheads.

Discussion of the obtained results

The CUDA results obtained can be explained by the structure of the convolution algorithm and by the characteristics of GPU execution. Since each output pixel can be computed independently, the problem is highly data-parallel and therefore well suited for GPU computing. This is the main reason why the CUDA version achieves much better performance than the CPU-based versions when the workload is sufficiently large.

For the smallest kernels, especially 5x5, the advantage of CUDA is less pronounced in the smallest images. In these cases, the amount of useful computation is relatively small, so the cost of launching CUDA kernels and transferring data between host and device has a stronger impact on the total execution time. As a result, the GPU still performs competitively, but the relative gain is more limited.

As both image size and kernel size increase, the behavior improves very clearly. Large kernels such as 49x49 and 99x99 require much more arithmetic work per output pixel, and large images increase the total number of pixels to process. Under these conditions, the computational cost of the convolution dominates the execution, and the transfer overhead becomes much less significant in relative terms. This explains why the CUDA implementation clearly outperforms the serial, OpenMP and hybrid versions in the largest and most demanding cases, especially for im04 and im05.

From the point of view of dependences, the CUDA strategy is very favorable. Each thread computes one output pixel and writes only to its own output position, while all threads read shared input data and kernel coefficients. Therefore, there are no write conflicts between threads, and the computation can be parallelized very naturally on the GPU. The main cost comes not from synchronization inside the kernel, but from memory transfers between CPU and GPU.

The comparison between CUDA configurations also shows that the implementation is relatively stable. The three tested block sizes (8x8, 16x16, and 32x8) provide very similar global behavior, although 16x16 and 32x8 tend to be slightly better in the most demanding cases. This suggests that block configuration does affect performance, but not as strongly as the overall problem size. For this reason, the most important factor in the observed CUDA speedups is the computational intensity of the workload rather than the difference between block shapes.

When compared with the previous deliveries, the CUDA version becomes the most advantageous solution in large workloads. The serial version is the slowest reference, and although the OpenMP and hybrid versions improve performance, they are still limited by CPU resources and, in the hybrid case, also by communication overhead. The CUDA version avoids MPI communication costs and benefits from much finer-grain parallelism, which is why it reaches the best execution times in the largest image and kernel combinations.

Overall, the results show that the CUDA implementation is especially effective for medium and large workloads, where the problem contains enough parallel work to fully exploit the GPU. In contrast, for the smallest cases, the benefit is lower because transfer and launch overheads have a larger relative impact on the total execution time.

Conclusions

In this delivery, we developed a CUDA version of the convolution program starting from the original serial implementation and focusing on the convolution phase as the main computational hotspot. The results obtained show that the GPU-based solution is especially effective for medium and large workloads, where it clearly outperforms the serial, OpenMP and hybrid versions. In contrast, for the smallest cases, the gain is more limited because host-device transfers and kernel-launch overheads have a larger relative impact on the total execution time. The comparison between CUDA configurations also shows that block size affects performance, although the overall problem size remains the dominant factor. Overall, the CUDA implementation proved to be the most favorable solution for the most demanding cases, confirming that the convolution problem is well suited for GPU parallelization.

CPU vs GPU computing

From our experience in this project, CPU and GPU computing differ not only in performance, but also in the way the programmer must think about the problem. In the CPU-based versions, especially in OpenMP, the programming model was more direct and easier to adapt from the serial code. The hybrid MPI+OpenMP version was more complex because it required explicit data decomposition, synchronization and communication between processes. However, the overall structure still remained close to the original program logic.

In contrast, the CUDA version required a different perspective. Instead of focusing on processes or CPU threads, the problem had to be expressed in terms of massive fine-grain parallelism, assigning GPU threads to individual pixels and considering host-device memory transfers as an essential part of the execution. From the programming point of view, CUDA was more difficult than OpenMP and also less immediate than the hybrid version, because even simple implementations requires explicit memory allocation on the GPU, copies between CPU and GPU, kernel launches, and configuration of thread blocks.

In return, the performance benefit of the GPU was much more noticeable in the largest and most computationally intensive cases. This makes GPU computing very attractive when the problem has enough data parallelism and enough work to amortize transfer overheads. Therefore, from a cost-benefit point of view, GPU programming is clearly worthwhile for heavy workloads such as large convolutions, although it also demands more effort from the programmer and a deeper understanding of the execution model.

Bibliography

1. **NVIDIA. CUDA C++ Programming Guide.**
Official reference for the CUDA programming model, including kernels, thread hierarchy, memory hierarchy, and GPU execution concepts.
<https://docs.nvidia.com/cuda/cuda-c-programming-guide/>
2. **NVIDIA. CUDA Runtime API Documentation.**
Official reference for the CUDA runtime functions used for memory allocation, memory transfers, kernel launches, and synchronization.
<https://docs.nvidia.com/cuda/cuda-runtime-api/index.html>
3. **NVIDIA Technical Blog. An Even Easier Introduction to CUDA.**
Introductory reference useful for understanding the organization of threads, blocks, grids, and the basic CUDA execution model.
<https://developer.nvidia.com/blog/even-easier-introduction-cuda/>